How Rules Work

This page provides you with an overview of how rules work to detect and prevent fraud with the resources that come out of the box with your application.

API response with Feedzai's recommendationâ

When a financial institution invokes Feedzai's API to score a request via the card authorization, transfer initiation or digital activity device intelligence endpoints, Feedzai sends an API response containing a decision recommendation. This field helps you decide whether the transaction is fraudulent or legitimate:

Behind the scenes, the event is processed by the risk engine platform, Pulse, which uses data science resources to evaluate the event. If you look at the result generated by rules, the API response contains a `decision` field with the recommendation values `approve`, `decline` or `review`, and the boolean `alert` field. The following section explains how both decision outcomes and alerts are generated.

How is Feedzai's recommendation generated?â

It is important to understand how rules process events and contribute to the API response to detect and prevent fraud efficiently. This section breaks down this life cycle in the following conceptual steps:

1. Mapping from API fields to schema fields
2. Rule options in the Pulse application
3. Rule blocks and outcomes in the Pulse workflow
4. Manual revision in Case Manager
5. API response

1. Mapping from API to Pulse schemaâ

The fields in the API request are mapped to a Pulse schema named *API Schema*. This schema is then extended with internal fields, including the ones that are needed to write the rule outcomes (e.g. `decision` field). This extension is named *Extended API Schema*. See the Map API Fields page to access the schemas and their associated fields.

The internal fields are used to register the outcome generated by a rule. At the end of the Pulse workflow, the decision rules generate a final outcome, which is then sent in the API response in the `decision` field. This concept is further explained in the following section.

2. Rule options in the Pulse applicationâ

Before getting into the specific logic behind each event, let's have a closer look at two important rule settings. Each rule is configured with **Actions** and **Mark As**:

When a rule with the action **Alert** is triggered, the event is displayed in the Case Manager alerts page for manual revision. This also sets the `alert` field as `true` in the API response.

In the **Mark As** option, you can select the outcome values that are configured beforehand in the Pulse workflow. When a rule is triggered, the **Mark As** value is written to the schema field that is associated with this rule outcome. For example, in this case, the **Decline** value is written in the `decision` internal field.

Different rules are defined with different outcomes according to the specificities of each event. The position of rules projects in the Pulse workflow, and the additional custom services dictate how the final recommendation is generated. This is further discussed in the next step.

3. Rule blocks and outcomes in the Pulse workflowâ

Each set of APIs contains a Pulse workflow (e.g. *Cards Workflow*, *Transfers Workflow*, *Digital Activity Workflow*). The possible rule outcomes and the rules' processing order are defined on each workflow:

How does the Cards Workflow generate the final outcome?

The *Cards Workflow* is configured with **decision**, **fraud**, and **sca_required** outcomes. The fraud outcome is generally configured to be used by machine learning models. This means that each rule contributes to the `decision` field, and the first rule triggering sets the final value. Learn more about the default rule precedence.

How do Transfers and Digital Activity Workflows generate the final outcome?

tip

The *Transfers Workflow* contains a slightly more complex logic:

- Transfers payment types
    - Traditional credit transfers (also called bank transfers or credit transfers)
    - Direct debits
    - Instant transfers (also known as instant payments or real-time payments)
- Rule's importance with a service enforcer to define priority (i.e. the default rule precedence does not prevail in this case)

Transfers and Digital Activity workflows are configured with the **decision**, **fraud**, **risk_level**, and **sca_required** outcomes. The latter is specific to financial institutions under the PSD2 regulation. Let's have a closer look at the conceptual logic behind the recommendation generation in transfers and digital activity. This logic comprises the following contributions:

The `risk_level` outcome contains the possible values **high**, **low**, and **medium**. Each rule contributes to the `risk_level` outcome, except the rules in the *Decision Rules* project, which contribute to the `decision` outcome. Therefore, when the event flows through the rules project preceding the *Decision Rules* block, rules write to the `risk_level` internal field.

Right before the *Decision Rules* block, the workflow includes an out-of-the-box custom service. When the event reaches this element, the custom service looks at the outcome value of each triggered rule (i.e. **high**, **low**, and **medium**) to enforce a new outcome. This new outcome is written to the `enforced_risk_level` field.

The custom service provides the following logic:

- **Rule order**: The service can be configured with a list of rules ordered by priority. The rule triggered with the highest priority defines the enforced outcome value.
- **Outcome value order**: The service can be configured with risk level order (e.g. medium - low - high). This means that the defined risk level order prevails over the order in which the rules triggered. In the medium - low - high example, if a rule with medium risk level is triggered, then this risk level is enforced.

Outcome Priority Enforcer Service in more detail

Let's assume the Outcome Priority Enforcer Service is configured in a workflow with the following configuration:

```
 pulse.global.services.outcomepriorityenforcer.output=enforced_risk_level
pulse.global.services.outcomepriorityenforcer.rules=WhitelistRuleId
pulse.global.services.outcomepriorityenforcer.order=high_risk,medium_risk,low_risk
```

To describe possible scenarios, let's use the notation `ruleId` , `outcomeValue` to indicate that the rule with ID `ruleId` triggered and set the outcome to `outcomeValue` .

Scenario 1: If an event triggers (`rule1` , `low_risk`) and (`rule2` , `medium_risk`), then the service will output `medium_risk` to the schema field `enforced_risk_level` . This happens because none of the rules is defined in the rule list, and *medium_risk* appears first on the order list, thus having higher priority.

Scenario 2: If an event triggers (`rule1` , `high_risk`) and (`WhitelistRuleId` , `low_risk`), then the service will output `low_risk` to the schema field `enforced_risk_level` . This happens because the rule `WhitelistRuleId` is configured in the rule's configuration property. Note that, in this scenario, the order's property is ignored since the rule configuration takes precedence over the order configuration.

Scenario 3: If an event triggers (`rule1` , `super_high_risk`), then the service will not output any value to the schema field `enforced_risk_level` . This happens because the service does not recognize `super_high_risk` as a possible value for both rule ID or outcome value.

Finally, the decision rules are defined with the **Decision** outcome. These rules look at the `enforced_risk_level` field and make a final recommendation as follows:

If	Then
`enforced_risk_level` is `high` OR `enforced_risk_level` is `medium` AND `payment_sub_type` is `instant_transfer`	`decision` field is set as `decline`

If	Then
`enforced_risk_level` is `medium`	`decision` field is set as `review`. The event is displayed on the alerts page in Case Manager
`enforced_risk_level` is `low`	`decision` field is set as `approve`

info

Instant transfers: For rules that act on the transfers payment method, only the decision block contains rules that can trigger an alert. Also, the Transfers - Decision Rules are defined by the order presented above. This means that instant transfers with `enforced_risk_level` field as `high` or `medium` are caught by the first rule and, therefore, automatically declined. This is important because instant transfers are not subject to manual revision due to their instant processing nature. Only credit transfers and direct debits can trigger an alert action which is later sent for review.

4. Manual revision in Case Managerâ

The following events generate alerts as follows:

- **Card events**: Various rules across rules projects contain the alert action. When one of these rules is triggered, the event is displayed in the Case Manager alerts page. See the rules' actions by checking each *Cards* rules project within the Rules Projects section.

- **Transfer events**: Only decision rules contain the **alert** action following the logic described in the previous section. Note that instant transfers is a payment type that is processed instantaneously. When these events are declined, they also generate alerts. They are manually reviewed with the sole purpose of giving feedback to the risk engine because the final recommendation has already been provided.

- **Digital events**: Rules track digital events to identify the user behavior during a banking session. If a digital event has signs of suspicious activity, it will trigger an alert which will be visible in Case Manager.

Analysts investigate alerts in Case Manager and decide whether the event is fraudulent or legitimate. This decision is registered in the `feedback_value` field as `true` or `false` for fraud and not fraud, respectively. Then, the *Feedback Rules* project evaluates this result and makes a new recommendation for card, transfer and digital events.

5. API responseâ

At this point, you already understand a scoring event's API response, which allows you to benefit from the recommendation provided by Feedzai. Note that, whether you send an API request via Cards, Transfers or Digital Activity streams using one of their scoring endpoints, you will get an API response with the following key fields:

- `decision` : The final outcome generated by rules according to the concepts explained in the Rule blocks and outcomes in the Pulse workflow section.
- `alert` : Whether the event generated an alert that is visible on the alerts page in Case Manager.
- `score` : The numeric value a machine learning model generates to evaluate an event. Possible values range from 0 to 1000.
- `action_codes` : Every rule has an array of Actions including the name of the rule (e.g. fraud rule Cards-FR1 contains a Cards-FR1 Action). The `action_codes` array includes all the Actions of triggered rules to further understand which rules have triggered for each event.
- `sca_required` : Strong Customer Authentication is required or not, based on compliance and risk assessment.

Learn moreâ

Check the Create and Manage Rules page to assist you in adding new rules to your Pulse Application. Additionally, to learn more about fraud scenarios or access a list of rules and metadata that are provided out of the box, check the following pages:

- Fraud Scenarios and Detection.
- Rules Projects.